

SKILL WEEK 4

Process Management and Shell Programming

Name: Balaji Patil

Roll Number: 2520080065

Tasks 1, 2 and 3 — Code and Output

TASK 1

CODE

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    pid_t pid;

    printf("Before fork():\n");

    printf("Process PID: %d\n", getpid());
    printf("Parent PID: %d\n\n", getppid());

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("CHILD PROCESS\n");

        printf("PID : %d\n", getpid());
        printf("PPID: %d\n", getppid());
        printf("State: Running\n");

        sleep(5);

        printf("State: Running\n");

        sleep(5);

        printf("\nChild process after sleep\n");

        printf("PID : %d\n", getpid());
        printf("PPID: %d\n", getppid());
        printf("State: Running\n");

        printf("Child process terminating...\n");
        exit(0);
    }
    else {
        printf("PARENT PROCESS\n");

        printf("PID : %d\n", getpid());
        printf("PPID: %d\n", getppid());
        printf("Child PID: %d\n", pid);
        printf("State: Running\n");

        printf("\nParent waiting for child...\n");
        wait(NULL);

        printf("Child process completed.\n");

        printf("Parent process terminating...\n");
    }

    return 0;
}
```

TASK 1 — OUTPUT

OUTPUT

```
balaji@BalajiPatil:~$ gcc task3.c -o task3
```

```
balaji@BalajiPatil:~$ ./task3
```

```
Before fork():
```

```
Process PID: 543
```

```
Parent PID: 342
```

```
PARENT PROCESS
```

```
PID : 543
```

```
PPID: 342
```

```
Child PID: 544
```

```
State: Running
```

```
CHILD PROCESS
```

```
Parent waiting for child...
```

```
PID : 544
```

```
PPID: 543
```

```
State: Running
```

```
Child process after sleep
```

```
PID : 544
```

```
PPID: 543
```

```
State: Running
```

```
Child process terminating...
```

```
Child process completed.
```

```
Parent process terminating...
```

TASK 2

CODE

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <time.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("Child Process Started\n");
        printf("PID : %d\n", getpid());
        printf("PPID: %d\n", getppid());

        printf("\nState 1: WAITING/SLEEPING\n");
        sleep(10);

        printf("\nState 2: RUNNING\n");

        time_t start = time(NULL);

        while (time(NULL) - start < 15) {
            volatile unsigned long long x = 0;
            for (unsigned long long i = 0; i < 100000000; i++) {
                x += i;
            }
        }

        printf("\nState 3: TERMINATING\n");
        exit(0);
    }
}
```

```
}

else {
    printf("Parent Process\n");
    printf("PID : %d\n", getpid());
    printf("Child PID : %d\n", pid);

    printf("\nParent is waiting for 35 seconds.\n");
    sleep(35);

    printf("\nParent calling wait().\n");
    wait(NULL);

    printf("Child process collected.\n");
    printf("Parent terminating.\n");
}

return 0;
}
```

TASK 2 — OUTPUT

OUTPUT

```
Parent Process
PID : 490
Child PID : 491

Parent is waiting for 35 seconds.
Child Process Started
PID : 491
PPID: 490

State 1: WAITING/SLEEPING

State 2: RUNNING

State 3: TERMINATING

Parent calling wait().
Child process collected.
Parent terminating.
balaji@BalajiPatil:~$ nano task4.c
balaji@BalajiPatil:~$ |
```

CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <ctype.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100

void expand_variables(char *input, char *output) {
    int i = 0;
    int j = 0;

    while (input[i] != '\0') {
        if (input[i] == '$') {
            i++;

            char variable[100];
            int k = 0;

            while (input[i] != '\0' &&
                  (isalnum(input[i]) || input[i] == '_')) {
                variable[k++] = input[i++];
            }

            variable[k] = '\0';

            char *value = getenv(variable);

            if (value != NULL) {
                for (int x = 0; value[x] != '\0'; x++) {
```

```

    }
    output[j++] = input[i++];
}

output[j] = '\0';
}

int parse_input(char *input, char *args[]) {
    int argc = 0;
    int i = 0;
    int length = strlen(input);

    while (i < length) {
        // Skip whitespace
        while (i < length &&
               (input[i] == ' ' || input[i] == '\t')) {
            i++;
        }

        if (i >= length)
            break;

        char *argument = malloc(MAX_INPUT);

        if (argument == NULL) {
            perror("malloc");
            exit(1);
        }

        int j = 0;

        // Single quotes
        if (input[i] == '\'') {

```

```
// Single quotes
if (input[i] == '\') {
    i++;
    while (i < length && input[i] != '\') {
        argument[j++] = input[i++];
    }
    if (i >= length) {
        free(argument);
        printf("Syntax Error: Unmatched single quote.\n");
        return -1;
    }
    i++;
}

// Double quotes
else if (input[i] == '"') {
    i++;
    while (i < length && input[i] != '"') {
        if (input[i] == '$') {
            i++;
            char variable[100];
            int k = 0;
            while (i < length &&
```

```

        char variable[100];
        int k = 0;

        while (i < length &&
               (isalnum(input[i]) ||
                input[i] == '_')) {

            variable[k++] = input[i++];

        }

        variable[k] = '\0';

        char *value = getenv(variable);

        if (value != NULL) {

            for (int x = 0;
                 value[x] != '\0';
                 x++) {

                argument[j++] = value[x];

            }

        }

        else {

            argument[j++] = input[i++];

        }

    }

    if (i >= length) {

        free(argument);

        printf("Syntax Error: Unmatched double quote.\n");
    }

```

```

        return -1;
    }

    i++;

}

// Normal argument
else {

    while (i < length &&
           input[i] != ' ' &&
           input[i] != '\t') {

        if (input[i] == '$') {

            i++;

            char variable[100];
            int k = 0;

            while (i < length &&
                   (isalnum(input[i]) ||
                    input[i] == '_')) {

                variable[k++] = input[i++];

            }

            variable[k] = '\0';

            char *value = getenv(variable);

            if (value != NULL) {

                for (int x = 0;
                     value[x] != '\0';
                     x++) {

```

```

                else {

                    argument[j++] = input[i++];

                }

            }

        }

        argument[j] = '\0';

        args[argc++] = argument;

        if (argc >= MAX_ARGS - 1)
            break;

    }

    args[argc] = NULL;

    return argc;
}

int main() {

    char input[MAX_INPUT];
    char *args[MAX_ARGS];

    while (1) {

        printf("myShell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strlen(input) == 0) {
            printf("Empty command.\n");

```

```
input[strcspn(input, "\n")] = '\0';

if (strlen(input) == 0) {
    printf("Empty command.\n");
    continue;
}

int argc = parse_input(input, args);

if (argc == -1)
    continue;

printf("--- Parsed Arguments ---\n");
for (int i = 0; i < argc; i++) {
    printf("Argument %d: %s\n",
           i + 1,
           args[i]);
}

if (strcmp(args[0], "exit") == 0) {
    for (int i = 0; i < argc; i++)
        free(args[i]);

    printf("Shell terminated.\n");
    break;
}

pid_t pid = fork();

if (pid < 0) {
    perror("fork");
}
```


TASK 3 — OUTPUT

OUTPUT

```
myShell> echo "Hello $USER"
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello balaji
Child Process
PID = 1254
Hello balaji
Child process completed.
```

```
myShell> echo 'Hello World'
--- Parsed Arguments ---
Argument 1: echo
Argument 2: Hello World
Child Process
PID = 1255
Hello World
Child process completed.
```

```
myShell> echo "Hello World
Syntax Error: Unmatched double quote.
```

```
myShell> exit
Shell terminated.
```

Note: The Task 3 output above is a representative output generated from the submitted parsing logic; process IDs can differ each time the program is run.